

Code:

```
GNU nano 8.7.1 escape_demo.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("Enter a command or text: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    printf("\n--- Parser Output ---\n");
    printf("Original Input : %s\n", input);

    printf("Escaped Space : ");
    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == ' ')
            printf("\\ ");
        else
            printf("%c", input[i]);
    }

    printf("\nSpecial Symbols: \$ @ # & ! * ?\n");
    printf("Escaped Quote : \"Hello World\"\n");
    printf("Escaped Backslash: C:\\\\OS\\\\Skill6\n");

    return 0;
}
```

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ ./escape_demo
Enter a command or text: hello world OS skill 6

--- Parser Output ---
Original Input : hello world OS skill 6
Escaped Space : hello\ world\ OS\ skill\ 6
Special Symbols: $ @ # & ! * ?
Escaped Quote : "Hello World"
Escaped Backslash: C:\\OS\\Skill6
```

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ echo Hello\ World
Hello World
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ echo "Hello World"
Hello World
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ echo 'OS Skill 6'
OS Skill 6
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ echo "OS & Linux"
OS & Linux
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ echo "File: *.c"
File: *.c
```

Child process code:

```

GNU nano 8.7.1 child_process.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process Created\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Child is executing 'ls -l'\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent is waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
        printf("Parent process completed.\n");
    }

    return 0;
}

```

Output:

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ nano child_process.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ gcc child_process.c -o child_process
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ ./child_process
Parent Process Started
Parent PID: 1654

Parent is waiting for child...

Child Process Created
Child PID: 1655
Parent PID: 1654
Child is executing 'ls -l'
total 44
drwxr-xr-x 2 uday uday 4096 Oct  7 03:38 SKILL6.c
-rwxr-xr-x 1 uday uday 16296 Oct  7 03:43 child_process
-rw-r--r-- 1 uday uday 793 Oct  7 03:43 child_process.c
-rwxr-xr-x 1 uday uday 16224 Oct  7 03:38 escape_demo
-rw-r--r-- 1 uday uday 728 Oct  7 03:38 escape_demo.c
Child process completed.
Parent process completed.

```

Child argument code

```
GNU nano 8.7.1 child_arguments.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <argument>\n", argv[0]);
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Argument received: %s\n", argv[1]);

        execlp("echo", "echo", argv[1], NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ nano child_arguments.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ gcc child_arguments.c -o child_arguments
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ ./child_arguments "Hello OS Skill 6"
Parent Process
Parent PID: 1693
Child Process
Child PID: 1694
Argument received: Hello OS Skill 6
Hello OS Skill 6
Child process completed.
```

Exec.c code:

```
GNU nano 8.7.1 exec_error.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    printf("Child process attempting to execute a program...\n");

    execlp("wrong_command", "wrong_command", NULL);

    perror("Execution Error");

    return 1;
}
```

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ nano exec_error.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ gcc exec_error.c -o exec_error
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ ./exec_error
Child process attempting to execute a program...
Execution Error: No such file or directory
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-06$ ./child_arguments "Linux Operating System"
Parent Process
Parent PID: 1740
Child Process
Child PID: 1741
Argument received: Linux Operating System
Linux Operating System
Child process completed.
```